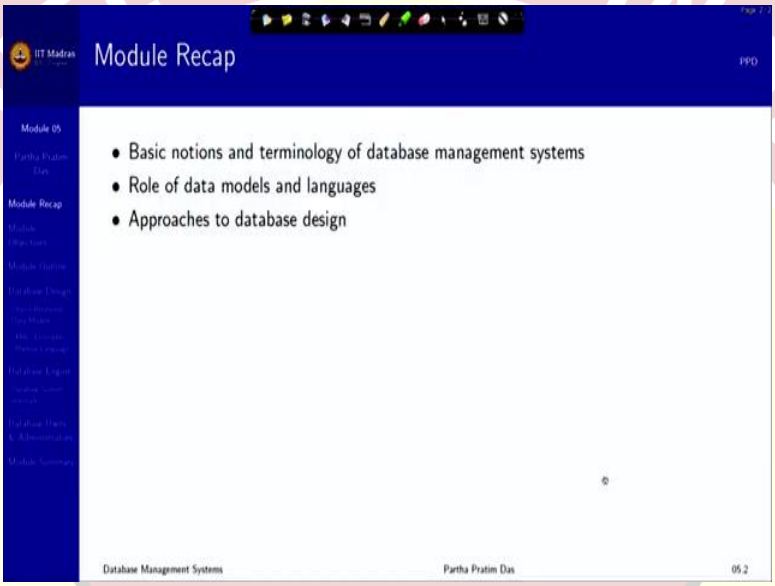
IIT Madras

ONLINE DEGREE

Database Management Systems
Professor Partha Pratim Das
Department of Computer Science and Engineering,
Indian Institute of Technology, Kharagpur
Introduction to DBMS/2

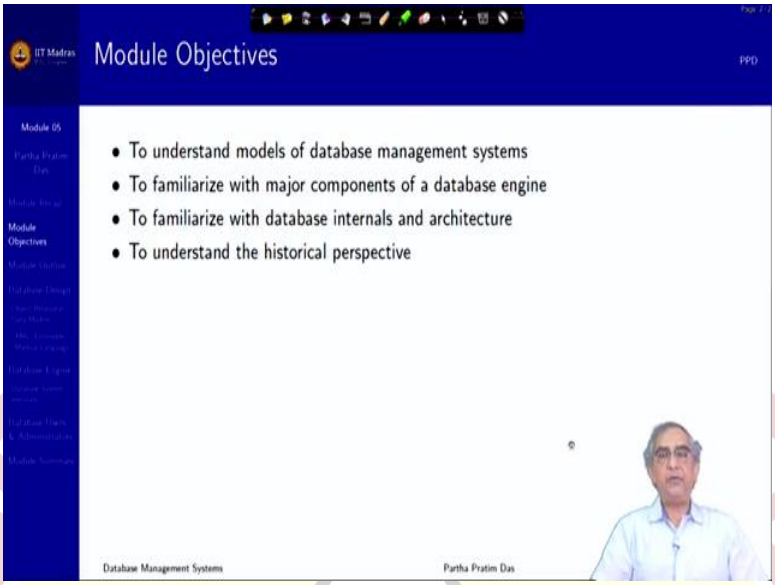
Welcome to module 5 of Database Management Systems course in IIT, Madras online B.Sc. program. We have in the last module started discussing about a overall basic introduction of the database management system.

(Refer Slide Time: 0:42)



So, specifically we have talked about the basic notions and terminology of a DBMS, the role of data models and languages in a very cursory manner, of course, not to any depth. The depth will come as we go forward. And we just get a, given a glimpse of what are the approaches to database design.

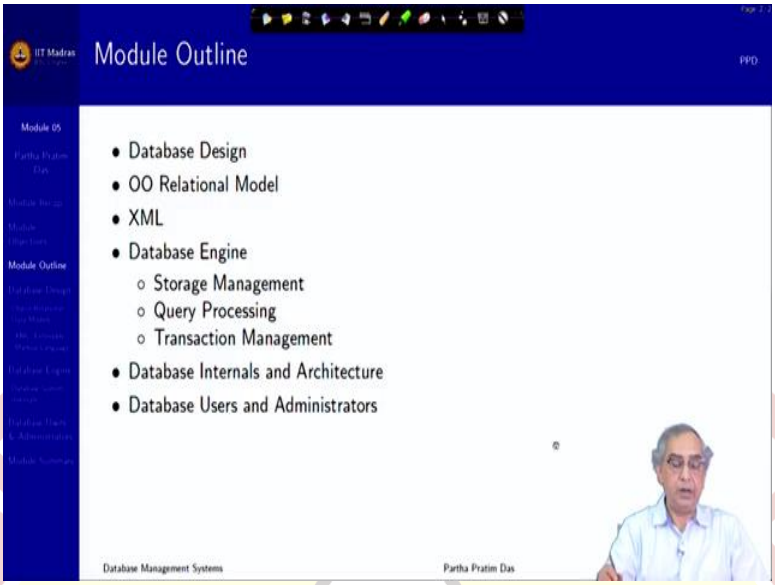
(Refer Slide Time: 1:04)



So, we will continue in this module five on the same theme and specifically we want to understand models of Database Management System little bit more and familiarize ourselves with what is not directly visible that is the database engine. We have talked about several factors. Why database systems are lot more capable than the file-based management of data through programming?

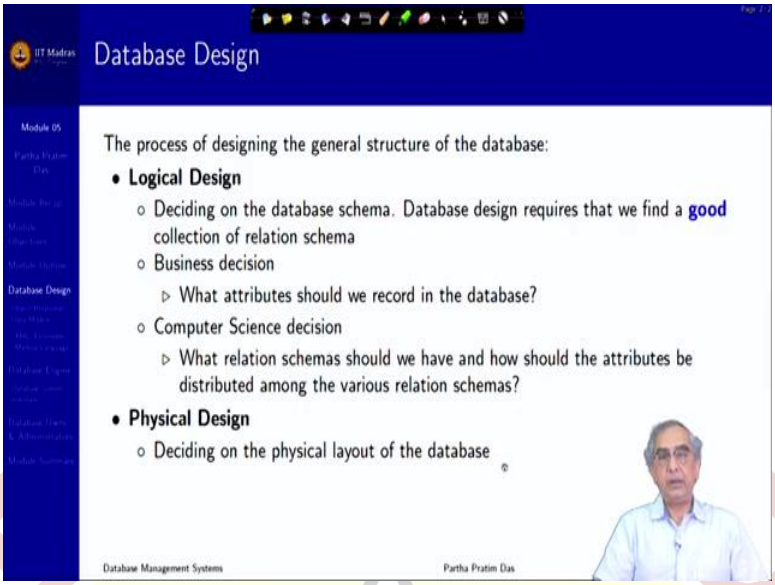
So, the key components which make that happen are parts of the database engine, so we will talk about that. We will talk very briefly about the databases internal and architecture, again all of these will be elaborated later and we will try to put that in the little bit of historical perspective. We have, of course, discussed the overall history at length.

(Refer Slide Time: 2:03)



So, these are the module outlines, which we will continue to see on the left banner as we progress.

(Refer Slide Time: 2:14)



So, talking about database design, we talked about it earlier too, this is more for continuity that I reiterate that a database design process, the general structure has two components; one is designing the physical database, which is how you physically lay out the data, the files, the B-trees, the pages in the desk, I am sorry, transactions with the memory and so on.

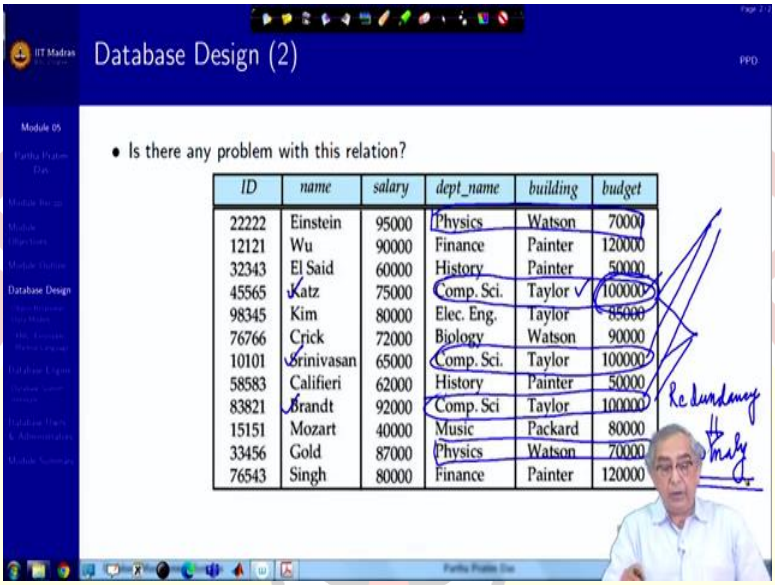
And the other which most of us will mostly deal with is a logical design, that is deciding on the database schema, how to organize the data, what should be the attributes, what should be the tables, what should be their relationships and so on. And this process of logical design is typically influenced by two stream of factors; one, which is true for every application is a business decision.

The businesses decide as to what information they want to keep, what kind of queries they want to put, so all those will dictate how we go about doing the logical design. The other stream of factors that need to be taken care of is in terms of, I say it is a computer science decision or the algorithmic decision where it talks about what is a relational schema that you should have.

Whether we should have it as a flat schema or we should have it at the segregated schema of multiple relations, how should we normalize it, we will need to know what normalization is, of course, so all these factors as applied to the business requirements given for the logical design. And what will often happen is physical design is done at an initial stage of the database system and typically remains invariant across different applications.

And I talked about earlier of physical data independence, so which makes sure that the logical design really does not need to know details about the physical design and if in future the physical design would have to change because of database migration or system upgrade, the logical designs will not get impacted; at least not significantly at all.

(Refer Slide Time: 4:49)



ID	name	salary	dept_name	building	budget
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

So, given these basic aspects let me get back to the question I had posed in the closing of the previous module, I am not sure whether you had time to look into this; is we presented a relation or a table to keep information about faculty, their affiliation to departments, their salary and the information about the department in terms of buildings they are housed in and the budget. And we said if we, of course, here all this information are available together.

So, the question is this a good design? Now, naturally you can understand that if I am asking the question obviously the answer is, it is not a good design and that is common sense. So, what is the problem that is particularly we need to know. So, let us try to observe something. Let us say let us look at this part, say computer science.

We have one entry here, we have another entry here, we have a third entry here, why do we have three entries, which are particularly identical; because we have three faculty, Katz, Srinivasan and Brandt, all of whom work for the computer science department. Now, obviously given the computer science department, the fact that it is housed in building Taylor cannot change across the faculty or its budget cannot change across the faculty.

So, what we have here is what I will say is redundancy. We have redundant data coming in here. The same data being repeated at multiple times and this is not specific for computer science, you can see that for physics here, you can see that for physics here, so as many faculty as a department would have, the data of the department, the building and budget data will get replicated.

Now, what is the problem with that? The problem naturally is quite evident, but less killing problem is the fact that the same data is being kept at multiple places, so you need more storage, so it is storage requirement. But a more severe problem is suppose anything changes, now say budget is not an invariant quantity, it keeps on getting changed, computer science does spending very fast in the time of data science and they get more budget allocated by the by the chairman of the university and then this quantity will have to change.

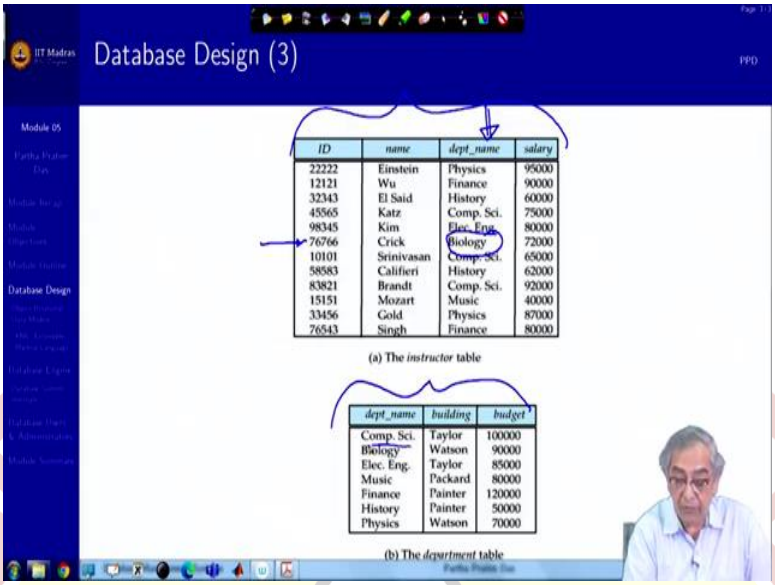
Now, the point is this quantity will have to change here, here, here, so as many faculty of computer science are there, this quantity has to be changed for all those rows, so this redundancy means that more work and the moment you have to change it at multiple places there is always that question of you may be, may have done a few and may have lost a few, may have missed a few.

There may be some system issues that happened after you updated two and you did not, could not update the third. So, anomaly will happen. So, redundancy means we will have anomaly. And this is dangerous for a database, if I have two entries now of computer science, where the budget figures are different naturally we have a deep, deep problem.

So, this is not a good design and we will say that we need to factor this in a way that only those information, which are relevant for a particular part of the data can be kept in one table, rest can be kept in a different table. So, what we see is what relates a faculty with the department or the building where the department is housed or what the budget of the department is; is the name of the department.

So, name of the department can tell you the budget and the building and given a faculty, knowing the name of the department we know everything about the department he or she belongs to, so instead of putting all these in a flat way, if we put it…

(Refer Slide Time: 9:38)



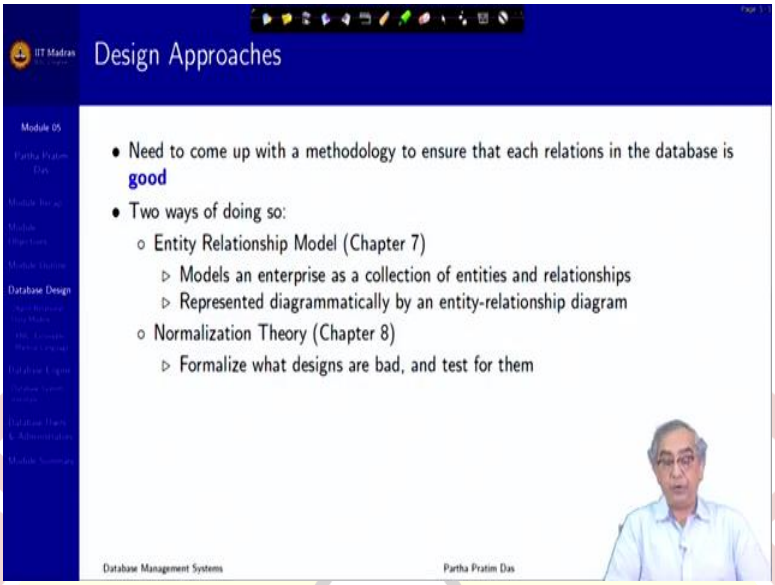
Let me go to the next slide, if we put it in this segregated form, where if you see here we just have the information of the faculty including naturally the department of affiliation and here we have only the information of the department not of the faculty, but so I can, if I want to know, say for professor Crick as to what is the budget of his department, I can always look up, this is biology, go to this table, look for biology and I will know the budget is 90,000.

The redundancy is gone, so is the chances of anomaly at the time of insertion, deletion or update. So, this is a basic principle. We will come to this and we will come to how we did this; here it is quite obvious, we are doing this by observation, but how we do this in general in a formal way is what we will talk, we will learn in the database design process.

So, what I wanted you to understand is one glimpse of what is the kind of design consideration that needs to go in and what is the kind of design output that we will prefer, so both of these designs; one is a flat table in the earlier slide and then this is a pair of two tables, in this both of them maintain the data, both of them will have all the information.

But there are serious reasons to prefer the two-table design over the one table one. But if we do that then it becomes an additional requirement to find out what is the budget of the department to which professor Einstein belongs, so we will have to also take care of that part, so these are the, some of the design code, design issues that we will talk about in this course. I am sorry.

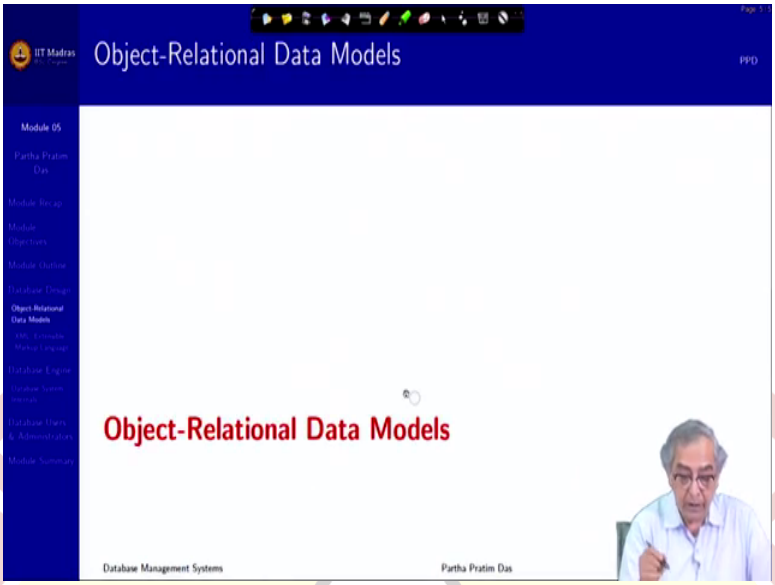
(Refer Slide Time: 11:44)

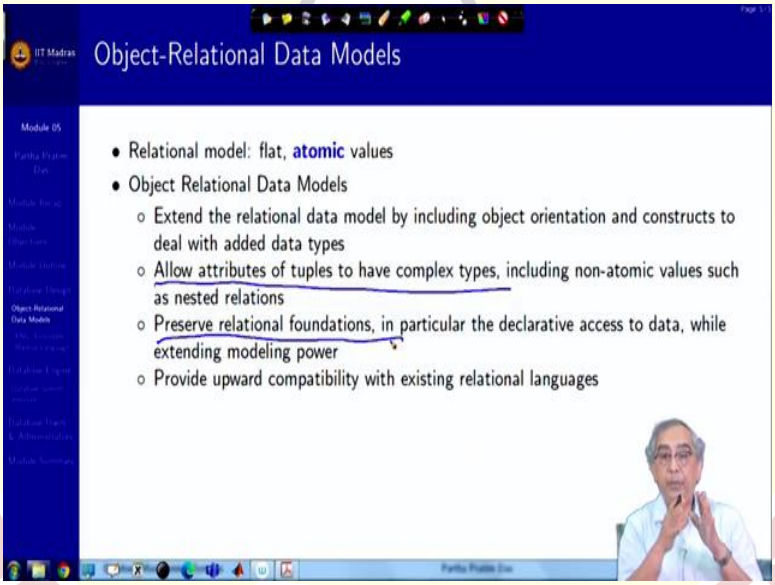


So, the design approaches for that therefore is to make sure that every relation in the database is good. There are two ways of doing that and primarily the two ways relate to the two sets of requirements we talked about. The first is the entity relationship model, which primarily tries to capture the business requirements, what are the entities, what are their relations, what are the attributes, what are their relationships, what is the kind of queries we want to answer and so on and put it in a diagrammatic form called the ER diagram which we will learn.

And then comes the question of how to organize that in terms of right set of tables in the relational model so that the problems like the one I just discussed does not arise, which is done through a formal theory that has been developed nearly 50 years, more than 50 years back, called the theory of normalization of tables or normalization of databases; which is particularly the computer science perspective.

(Refer Slide Time: 12:51)





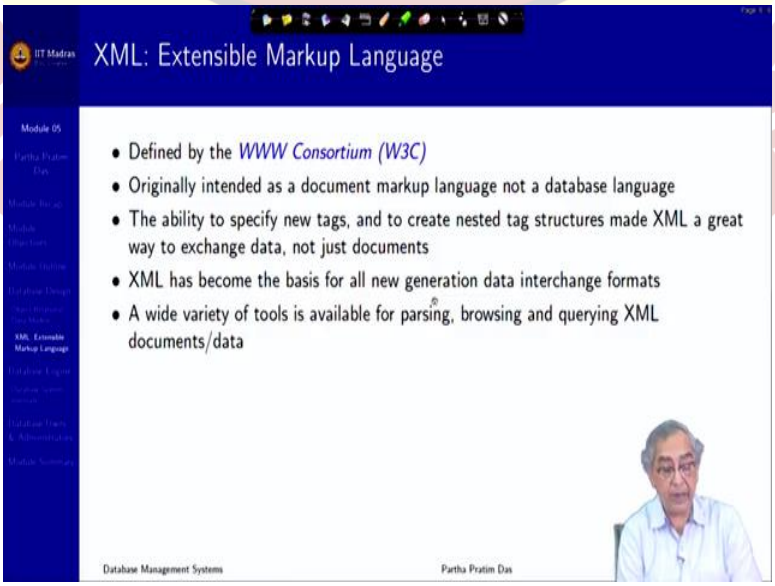
Moving on you may note that besides relational, which is primarily what we are going to discuss there are object relational data models where in a relational model every value is flat, atomic value, like it is a string or a number or any every attribute is either a string or a number or a character or a date, but it cannot be actually a composite thing, like an attribute could not be a student, because it has multiple other components, composite components.

Something which the object-oriented paradigm talks of very frequently that you can take any arbitrary collection of information and club them together into a one entity called the object and that becomes unitary, that becomes atomic, that particular facility or that particular model does not exist in the relational view, which is flat tabular with atomic values.

So, object relational data model is an extension of the relational data model which allow attributes, so these are some of the key things you can say it allows attributes to have complex types complex is not in the sense of complex numbers, but complex in terms of the complexity which are non-atomic values or composite values and, but preserves the basic relational foundation.

So, these are little bit more advanced but has been in the community for quite some time. We will try to give you a glimpse if we get time, but primarily I just want you to know that what you are learning in relational database is key, but it is not the only thing.

(Refer Slide Time: 14:39)

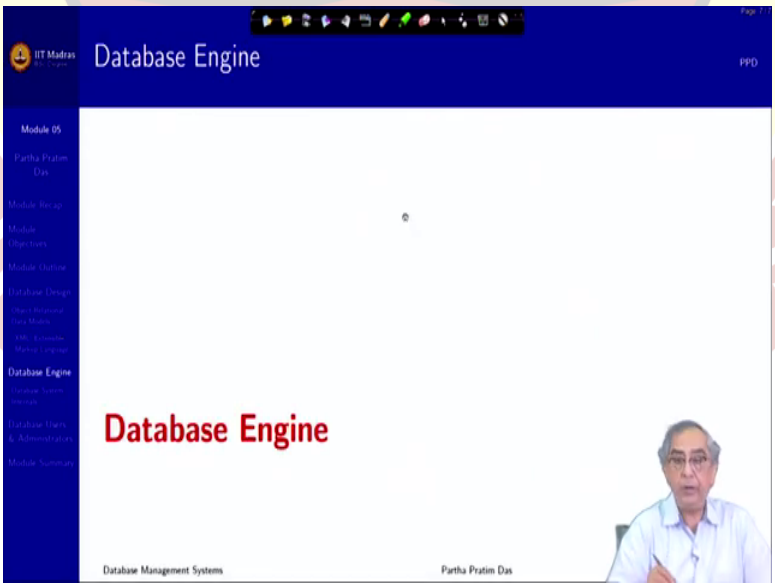


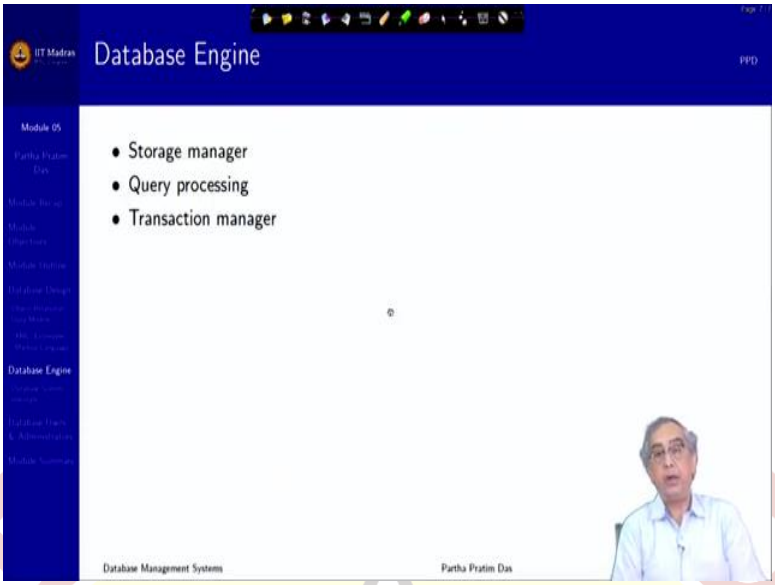
The other which has become or I should say in parallel what has become very-very popular and is very useful is the XML - Extensible Markup Language defined by the W3C as it is called, WWW consortium. It was originally designed for marking up documents, so what it, what XML primarily says; XML is a description of a name value pair. So, it talks about a tag, so that you can put a value on that.

But the interesting part is the tags could be composite, a tag could have in a part of it multiple other tags and they itself, themselves could have other values; so, it is possible to have a hierarchical description in XML, which is not possible enough in any kind of a flat representation and this has become an extensively used format or used language, I mean, it is not a language in this computational sense.

It is a language in the descriptive sense and it has become very popular particularly for the purpose of data interchange like you have to take a dump of a database and keep it in the store outside of the database system, most often people will put it in XML and then later on you can import it back to the database system and so on, and we will see that many of the newer database systems which are non-relational, may be some of which deal with unstructured data also makes use of the XML as their backbone for representation.

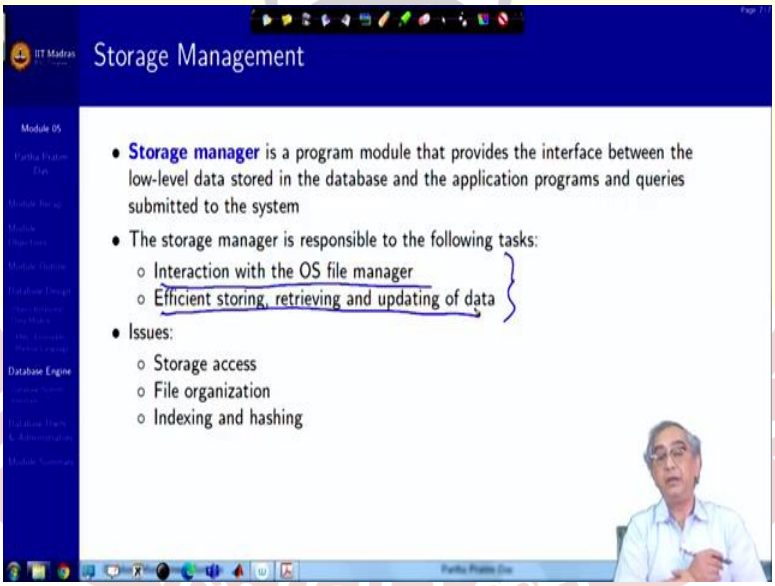
(Refer Slide Time: 16:25)





So, going on to the database engine which is the core of what supports the whole activity; there are three major components, one is a storage manager, query processing and transaction manager. Naturally three of them serve three different aspects of the whole problem.

(Refer Slide Time: 16:41)

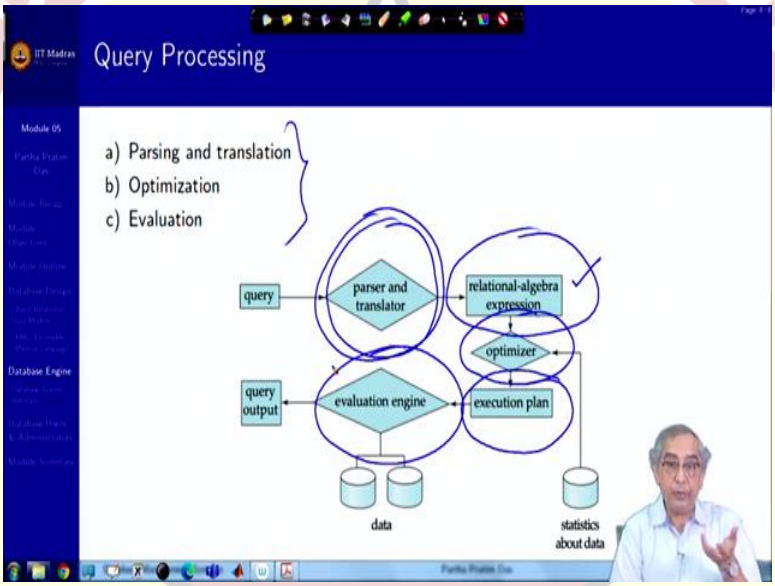


Obviously, the database is a store, so you need a storage manager, which will store the entire data of the database, so we talked about the physical design, we talked about data dictionary, all of these are factors which the storage manager has to deal with; it is responsible to note these two primarily that it has to interact with the operating system file manager, because finally in the system you have a file manager to keep everything.

You are not creating a new computing system; you are developing the database system over on top of what you have in the computing systems file manager. So, somewhere there has to be a link of how you keep it in terms of files, how these files are indexed, how those files are accessed and so on and store manager, storage manager primarily takes care of those.

And it has to in that process make it efficient for storage, it must use as less storage as possible, it must be efficient to retrieve, it must be efficient to update the data. So, these are some of the issues, the access organization, indexing, hashing and while we talk about the store management, we will talk about all of these factors and it will become clearer.

(Refer Slide Time: 18:01)



The second of course, if the data is there we need to query the data naturally that is the basic purpose, we need to know what is there. So, that process has three primary parts; one is parsing and translation; that is we are writing that query in terms of some language. I have given you glimpses of the SQL language, so we will have to translate that, the SQL as I said is a commercial query language.

We have to translate that to the mathematical model because that is what is finally implementable. So, you need a way to understand the SQL that is parsing and you need a way to convert that into the relational algebra expression, which are more like algebraic equations, so that is what is the first step that is done in query processing, then you have it all in terms of relational algebra.

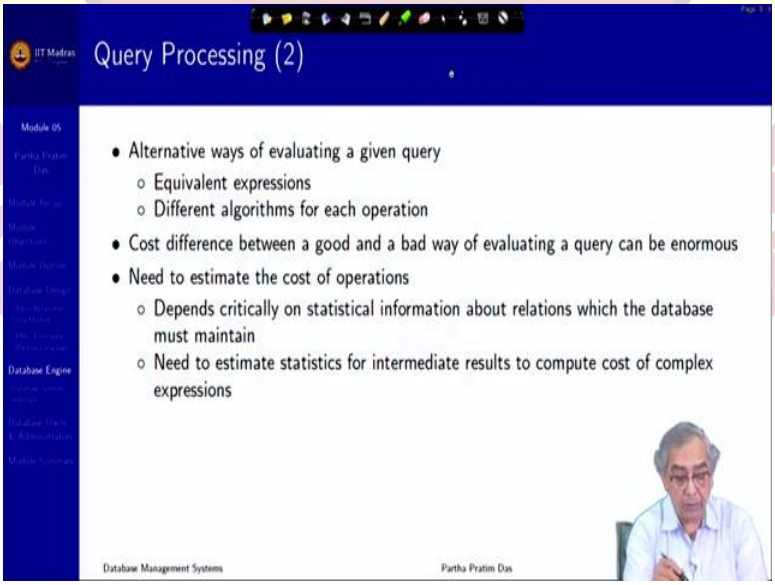
Once it comes to that you can use a lot of theory of relational algebra to optimize the query, to make sure that you have, you find out the answer in optimal number of steps. You know in algorithms they are the same problem can be solved through different algorithms; some are more efficient some are less efficient, so exactly the similar concept applies here.

You optimize and then you decide on an execution plan that as to how you will actually access the data, access the tables and different fields and get that answer out and as you do that you evaluate how you are doing, because the performance of your optimized query or a particular query is not absolute, it is not like a sorting problem where you just talk about the best case, worst case, average case and so on.

But here the performance depends on what kind of data distribution you actually have, so you might need to change the way you handle future queries depending on how you have done in this query. So, that is the purpose of the evaluation engine, which keeps the statistics of how you have done, what is good, what can be more improved and so on.

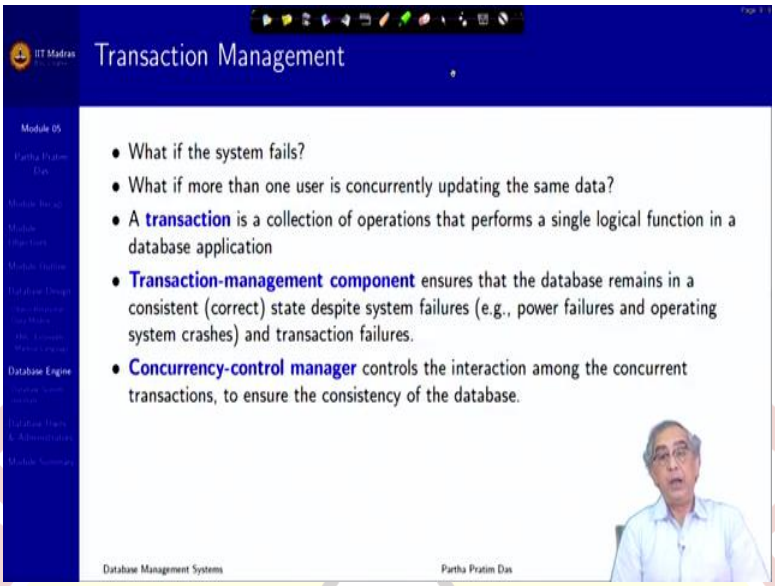
And that is what gets generated from the evaluation engine and you can see that the optimizer is trying to use that statistics and finally you have the query output. So, this is the whole purpose of the second major component of the database engine which is query processing.

(Refer Slide Time: 20:34)



Now, these are these are all details that we have, I have just talked about this is for your reference.

(Refer Slide Time: 20:41)



Now, the third and probably the most interesting part is the transaction management. We have taken examples of transaction. What is the transaction? It is a collection of operations that perform a single logical function, so we said that the transaction is making a payment from my account for 100 rupees to my friend.

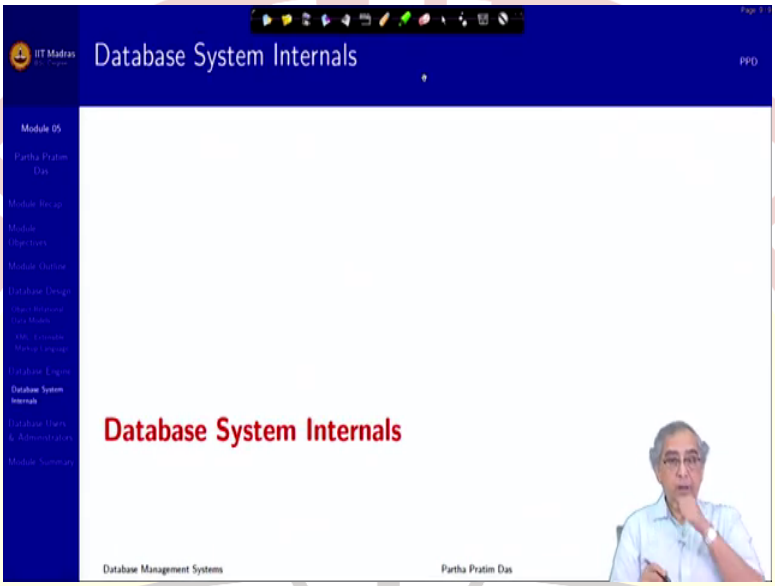
And that involves multiple operations like checking my account validity, checking my account balance, checking my friends account validity, checking after all these checks pass, then really debit my account, then take that money credit his account, update these in the transaction logs and so on so forth so, this whole thing together is a transaction till you do the commit, you remember we talked about commit.

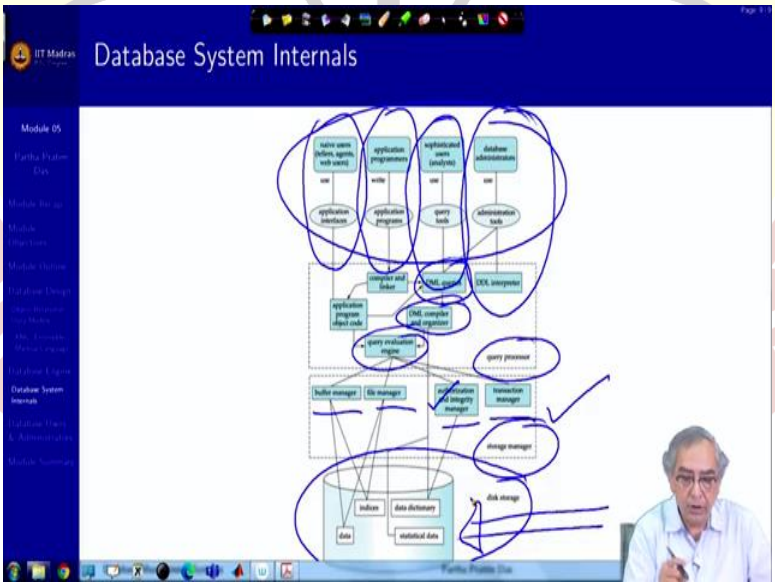
Commit is you are committing, no this is done, so that transaction management is a regular run time process that will happen, which will make use of queries, which will make use of the storage subsequently, so it is the three layers as you can see is building up, at the bottom there is store manager which is allowing you to interact with the data.

Then is a query manager which is allowing you to ask specific queries or request for specific updates and then there is a transaction manager, which allows you to do a business operation in terms of one or more queries being fired on the database system. Then there are issues of concurrency as to how does multiple people address the access to the database at the same time, how to serialize that.

That is if two people try to buy the same birth on a train, naturally that cannot be allowed, so in some way as you see in the operating system the actions will have to be properly serialized so that either A gets the birth or B gets the birth, but not both of them get the birth, neither it should be that none of them get the birth. So, these are the issues of the transaction management.

(Refer Slide Time: 22:38)





In the internals I will be very sketchy because it will take quite some time for you to understand the internals, but you can broadly see we have talked about storage management. So, what you see here at the bottom is a physical storage and this is a storage manager and these are the some of the sub-components of the storage manager.

For managing the storage, it has to manage the file it has to manage some buffers between the file system and the actual runtime, execution, it has to manage the authentication and so on; then you have the query processor which has to do, I mean, the kind of steps that we have talked about, the DML queries, it has to do optimization, it has to do evaluation.

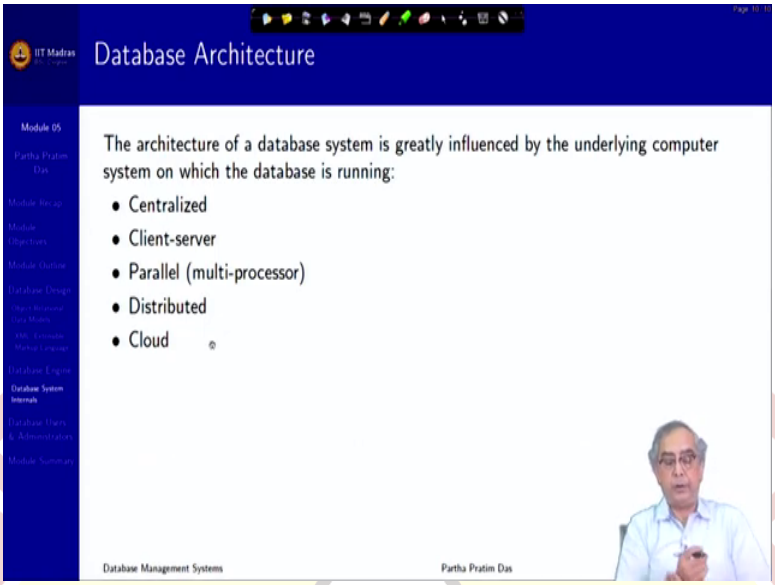
We have talked about these and on top of that here you have the different transaction management tasks where the naive users or the simplest users will use an application interface, like I am trying to buy say a bag of 1 kilo of potato from Big Bazaar. So, I am a naive user. I am using the application interface of the Big Bazaar e-commerce portal to buy that. Then there are application programmers who have actually written that portal.

Who have actually written all those put to, I mean, search for the products, add to check the details, add to cart, set the address, check out, do the payment, finally confirmation of payment and all that, so all those have been written by those. Some of the some of the people in this will be more advanced people, so those are called analysts.

So, they are like gurus of the application programmers, who look more deeply into complex queries, build specific tools for certain queries which are frequent, you can understand that in the in an application like e-commerce, like Big Bazaar or it is Snapdeal or is Flipkart, a similar type of query will come in million times, so you need to specifically look into how to organize that query, how to convert that from the web interface into the SQL form so that the optimization can be the best.

And finally, you have the database administrators here who administer the whole thing, who sets up this storage manager, who sets up the physical store, which configures different parameters of the database, which does the installation, takes care of the regular backup and recovery issues and so on, so they are the supremo of the database management system. So, this is a roughly the very broad internals of a database system.

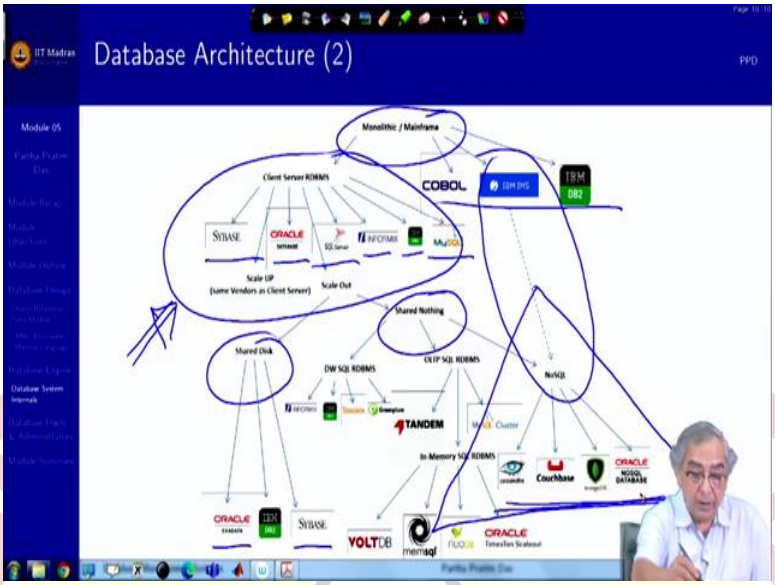
(Refer Slide Time: 25:36)



So overall, this system is been designed in multiple different architecture. Initially, naturally when it started it was centralized, there was one store; that store was necessarily, as I said was using some magnetic tape or something, the language or Cobol is more a file-based system, then you had relational systems, which are centralized at one place everything was happening, then came the concept of client server.

So, the database becomes a server, your applications are clients, so you have a loosely coupled system. Then you started parallelizing it, so that you can have multiple clients and multiple servers interacting in between them in a multiple different ways, then you could distribute it physically over the local area network, wide area network, over the Internet and so on and what you have now is you can put things in the cloud.

(Refer Slide Time: 26:32)



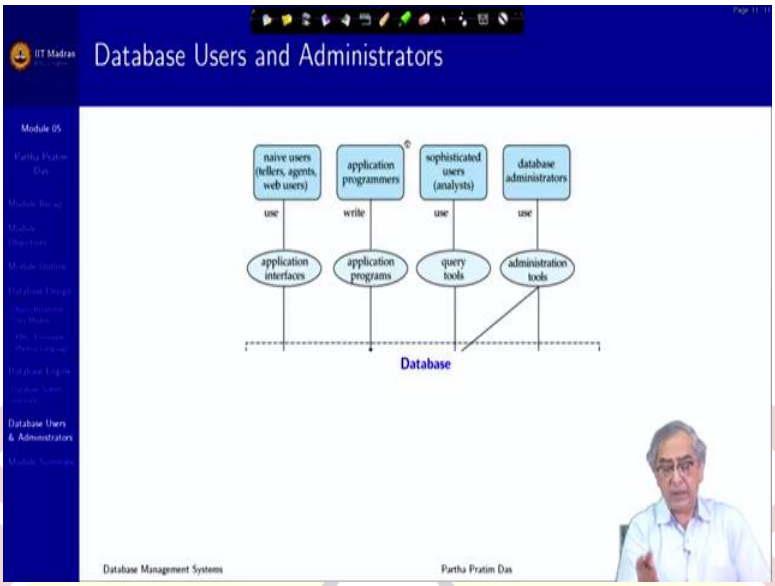
So, there has been a major evolution of this architecture. You saw this diagram earlier too, I just brought you back here after little bit more discussion, so this is the monolithic main frame. I am sorry; the monolithic mainframe is what was the earliest stage where you had Cobol and all that. And then you have client server RDBMS, Sybase, Oracle, SQL server, Infomax, DB2, MySQL, PostgresSQL, there are several others.

And primarily this is, this client server-based model is what we are going to focus primarily on, because rest of the models are built on top of this client server kind of model. So, you have a shared disk model, you have shared nothing model and so on, so that you can scale out. There are more advanced versions of Oracle and Sybase and all.

That which we will just mention; but we will not have enough time or preparedness in this course to talk in depth of those, our primary focus will be in terms of these architectures which are client server loosely coupled database architecture, which can be easily put to parallel systems, distributed systems and can be made available on cloud.

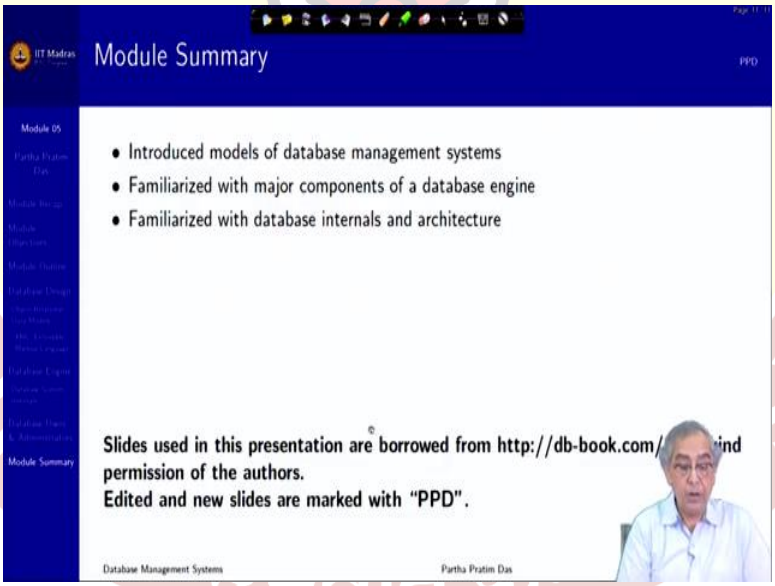
The other section is this part which is the architecture of unstructured, architecture for unstructured data, which is primarily the NoSQL paradigm, where very interestingly some of the old systems, particularly the network system found quite a bit of use. So, they have been you know repurposed, they have been reused with modification and there are several, very interesting unstructured databases are coming up in terms of the NoSQL paradigm, which will be good to know if you are particularly focusing on becoming an expert data scientist.

(Refer Slide Time: 28:37)



So, database user and administrators I do not think I need to explain anything further here, I have already explained it as a part of that.

(Refer Slide Time: 28:43)



So, I will conclude the module here where I have introduced the models of database management systems and familiarized you with the major components of the database engine and little bit of the overview of the architecture that a database system might have. Thank you very much for your attention and see you in module 6 next week.